

Chaos Lab Build Guide & Operator's Manual

Generated: 2026-02-20

Absolutely – here's a single "Build Guide + Operator's Manual" you can drop into your repo as:

quantum-chaos-enterprise-lab/CHAOS_LAB_BUILD_AND_OPERATIONS.md



Chaos Lab Build Guide & Operator's Manual

Purpose

Chaos Lab is a simulated enterprise environment for validating crypto discovery, TLS posture, certifi

It's designed to support:

- Quantum readiness assessment testing (PQC transition planning signals)
- Service inventory enrichment (protocol classification + non-TLS services)
- Certificate scenario coverage (expired/self-signed/missing chain/weak keys)
- Enterprise-ish topology (ingress + SNI vhosts, identity + PKI, cloud simulators)
- Operational realism (cert rotation automation)



What's Included

Core (always-on)

- TLS endpoints: modern, legacy, expired, self-signed
- mTLS-required endpoint (handshake/client-cert failure scenario)
- Plain HTTP endpoints: 8000, plus HTTP on a "TLS-ish" port 8444
- SSH on non-standard port 2222
- Unknown TCP services 5555 (+ optional 5556)
- Slow handshake proxy (latency simulation): 12443

Phase A (fast ROI for reporting)

- More service types (datastores/mgmt ports/etc.)
- TLS chain scenarios: missing intermediate, RSA 1024, SHA1
- Ingress/SNI: multiple vhosts behind one TLS port

Phase B (cloud simulators)

- LocalStack (AWS emulation): S3 + STS + IAM behind TLS + SNI (aws.chaos.local)
- Azurite (Azure emulation): blob/queue/table behind TLS + SNI
- blob.chaos.local, queue.chaos.local, table.chaos.local

Phase C (identity & PKI)

- Keycloak + Postgres (identity)
- step-ca (local PKI issuing short-lived certs)
- mTLS gateway using step-ca-issued certs (Phase C gateway)
- Cert rotation automation script (re-issues and reloads gateway)



Prerequisites

Required

- Docker Desktop (macOS or Windows)

- Docker Compose v2 (docker compose ...)
- openssl on host (macOS has it; Homebrew recommended if needed)

Recommended

- curl
- git
- A terminal that supports long-running processes (rotation loop)



Repo Layout

Typical layout (key paths):

- docker-compose.yml – main lab orchestration
- certs/ – base TLS materials used by core services
- certs/scenarios/ – cert scenario materials (Phase A2)
- certs/stepca/ – step-ca derived artifacts (Phase C)
- nginx/ – nginx configs per module (core / phaseA / identity / cloud)
- haproxy/ – slow handshake proxy config
- scripts/ – operational scripts (Phase C issuance/rotation)



Build Guide

1) Validate compose file parses

From the lab root directory:

```
docker compose -p chaoslab -f docker-compose.yml config >/dev/null && echo "■ compose OK"
```

2) Start core lab

```
docker compose -p chaoslab -f docker-compose.yml up -d
```

3) Start with profiles (recommended)

Start everything you've built so far:

```
docker compose -p chaoslab -f docker-compose.yml \
  --profile phaseA --profile cloud --profile identity --profile pki up -d
```

Notes

- identity enables Keycloak, step-ca, LDAP stack.
- pki enables the Phase C mTLS gateway.
- cloud enables LocalStack/Azurite + TLS fronts.
- phaseA enables expanded services + chain scenarios + ingress/SNI.

4) Confirm running services and ports

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```



Operator's Manual

Common Operations

Check lab status

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

View logs for a service

```
docker logs chaoslab-tls-modern-1 --tail 100
```

Restart a single service

```
docker compose -p chaoslab -f docker-compose.yml restart tls-modern
```

Full stop (keep volumes)

```
docker compose -p chaoslab -f docker-compose.yml down
```

Full reset (remove volumes)

■■ This deletes persistent data (Keycloak DB, step-ca home, etc.)

```
docker compose -p chaoslab -f docker-compose.yml down -v
```

■

Phase C (PKI / mTLS) Operations

What Phase C does

- step-ca runs as a local CA and issues short-lived materials.
- scripts/phaseC_stepca_issue.sh:
 - Generates server cert + key
 - Generates full chain server cert (mtls-stepca-fullchain.crt)
 - Generates client cert + key
 - Generates client chain (client1-chain.crt)
 - Generates CA trust bundle (ca-bundle.crt)
- Restarts mtls-stepca-gateway by default so the served cert rotates

Phase C script usage

One-shot issue + restart gateway

```
chmod +x scripts/phaseC_stepca_issue.sh
./scripts/phaseC_stepca_issue.sh
```

Rotation loop (every 5 minutes)

```
./scripts/phaseC_stepca_issue.sh --loop --every-min 5
```

Do not restart gateway (testing only)

```
./scripts/phaseC_stepca_issue.sh --recreate 0
```

Validate Phase C mTLS behavior

Expected failure (no client cert)

```
curl --cacert certs/stepca/ca-bundle.crt \
  --resolve mtls-stepca.chaos.local:17443:127.0.0.1 \
  https://mtls-stepca.chaos.local:17443/__tls_ok -v
```

Expected: HTTP 400 / "No required SSL certificate was sent"

Expected success (client cert chain)

```
curl --cacert certs/stepca/ca-bundle.crt \  
  --cert certs/stepca/client1-chain.crt \  
  --key certs/stepca/client1.key \  
  --resolve mtls-stepca.chaos.local:17443:127.0.0.1 \  
  https://mtls-stepca.chaos.local:17443/__tls_ok
```

Expected output: OK

Verify the served server cert (should rotate)

```
openssl s_client -connect 127.0.0.1:17443 -servername mtls-stepca.chaos.local </dev/null 2>/dev/null  
| openssl x509 -noout -dates -serial
```



Cloud Simulator Operations (Phase B)

LocalStack health (through TLS front)

```
curl -k --resolve aws.chaos.local:24566:127.0.0.1 https://aws.chaos.local:24566/__tls_ok  
curl -k --resolve aws.chaos.local:24566:127.0.0.1 https://aws.chaos.local:24566/_localstack/health
```

Azurite TLS fronts

```
curl -k --resolve blob.chaos.local:21000:127.0.0.1 https://blob.chaos.local:21000/__tls_ok  
curl -k --resolve queue.chaos.local:21001:127.0.0.1 https://queue.chaos.local:21001/__tls_ok  
curl -k --resolve table.chaos.local:21002:127.0.0.1 https://table.chaos.local:21002/__tls_ok
```



Ingress/SNI Operations (Phase A3)

Confirm vhost routing works (example)

If ingress uses SNI + host-based routing, test with --resolve:

```
curl -k --resolve app1.chaos.local:24443:127.0.0.1 https://app1.chaos.local:24443/  
curl -k --resolve app2.chaos.local:24443:127.0.0.1 https://app2.chaos.local:24443/
```

(Your exact hostnames depend on your nginx/phaseA/ingress/nginx.conf vhosts.)



Troubleshooting Guide

“Conflict. container name already in use”

Cause: old containers exist from previous project name / manual runs.

Fix:

```
docker rm -f <container_name>
```

Or reset:

```
docker compose -p chaoslab -f docker-compose.yml down -v
```

Port already in use

Cause: a local service is binding the same port.

Fix options:

- Stop the conflicting process
- Change the host port mapping in compose
- Use a different compose project name:

```
docker compose -p chaoslab2 -f docker-compose.yml up -d
```

"depends on undefined service" when using only one profile

Cause: profile excludes dependencies.

Fix:

- Use combined profiles (e.g. `--profile identity --profile pki`)
- Or remove `depends_on` for cross-profile-only validation commands

Cert rotation "runs" but served cert doesn't change

Cause: gateway wasn't restarted.

Fix:

- Ensure script runs restart block
- Or manually:

```
docker compose -p chaoslab --profile identity --profile pki up -d --force-recreate mtls-stepca-gate
```

curl fails with "unable to get local issuer certificate"

Cause: server chain not presented or client trust is incomplete.

Fix:

- Use `--cacert certs/stepca/ca-bundle.crt`
- Ensure nginx serves `mtls-stepca-fullchain.crt`



Security Notes

This is a local lab. Still:

- Avoid exposing these ports to the public internet.
- Keep Docker Desktop restricted to local interfaces.
- Treat generated private keys as sensitive (even in lab contexts).



Versioning Notes

Suggested version tracking:

- "Chaos Lab v1": core + phaseA baseline
- "Chaos Lab v2": cloud + identity + step-ca
- "Chaos Lab v3": Phase C gateway + rotation automation



Next: Enterprise Realism (planned)

When we resume:

- Additional clients (client2/client3)
- CN/OU-based allowlist/denylist
- Break-glass cert role (shorter TTL)
- Scanner/report outputs: "mTLS required", "issuer chain", "rotation detected"



If you want, I can also:

- generate this as a PDF (for sharing with stakeholders), or
- create a README.md + docs/ structure with separate pages (Build / Operations / Troubleshoot)